Shahariar Nadeem

CSE 4300

Design Documentation

This project was about building a C program that mimics how an operating system manages memory when there is not enough space for every page. The program takes a list of page numbers, a reference string, and a fixed number of memory frames, then tries to handle those pages using four different replacement strategies like FIFO, Random, LRU, and MIN, which is the optimal method if you know future requests. As it walks through the list, it keeps track of how many times a requested page is already in memory, a hit, and how many times it has to bring in a new page, a fault. When it finishes, it prints a short summary of the results and a detailed, step by step table that shows what happened at each step.

To keep things organized, the code is split into multiple files. The header file, page_replacement.h, defines a small structure called SimResult that stores the number of hits and faults, and it lists the functions used for running the simulations and traces. The main logic is in page_replacement.c. That file holds the helper functions, the actual implementations of the four algorithms, and the code that builds the trace output. For example, init_frames fills the frame array with -1 to mark empty slots, and find_page checks if a requested page is already sitting in one of the frames.

All four algorithms follow the same basic pattern. Each one takes the page list, the number of pages, and the number of frames as input. It sets up a result structure, allocates an array for the frames, and then processes the page requests one by one. If a page is already in a

frame, it counts as a hit. If it is missing, that is a fault, and the algorithm decides where the new page should go. It uses any empty frame first. Once all frames are full, each method follows its own rule to decide which page to remove. When the algorithm is done, it frees the memory it used and returns the result.

Here is a quick overview of how each method works

- FIFO replaces the page that has been in memory the longest.
- Random picks a frame at random to replace, so the outcome can change from run to run.
- LRU keeps track of when each page was last used and removes the one that has not been used for the longest time.
- MIN looks ahead at the remaining page requests and removes the page that will not be needed for the longest time, or will not be used again at all.

To make the behavior easier to see, each algorithm also has a matching trace version. These trace functions perform the same steps as the main simulations, but after each page request they print a row in a table. The table shows the current step number, the page requested, what is currently stored in each frame, and whether that step was a hit or a fault. When the trace finishes, any memory that was allocated for it is released.

The main driver of the program is in page_simulator.c. This file is where execution starts. It seeds the random number generator so the Random method behaves differently every time. Then it asks the user how many frames to use and how many page references they want to enter. It checks that this input is valid, reads the reference string into an array, and handles any problems such as bad input or memory allocation failure by printing an error and exiting safely.

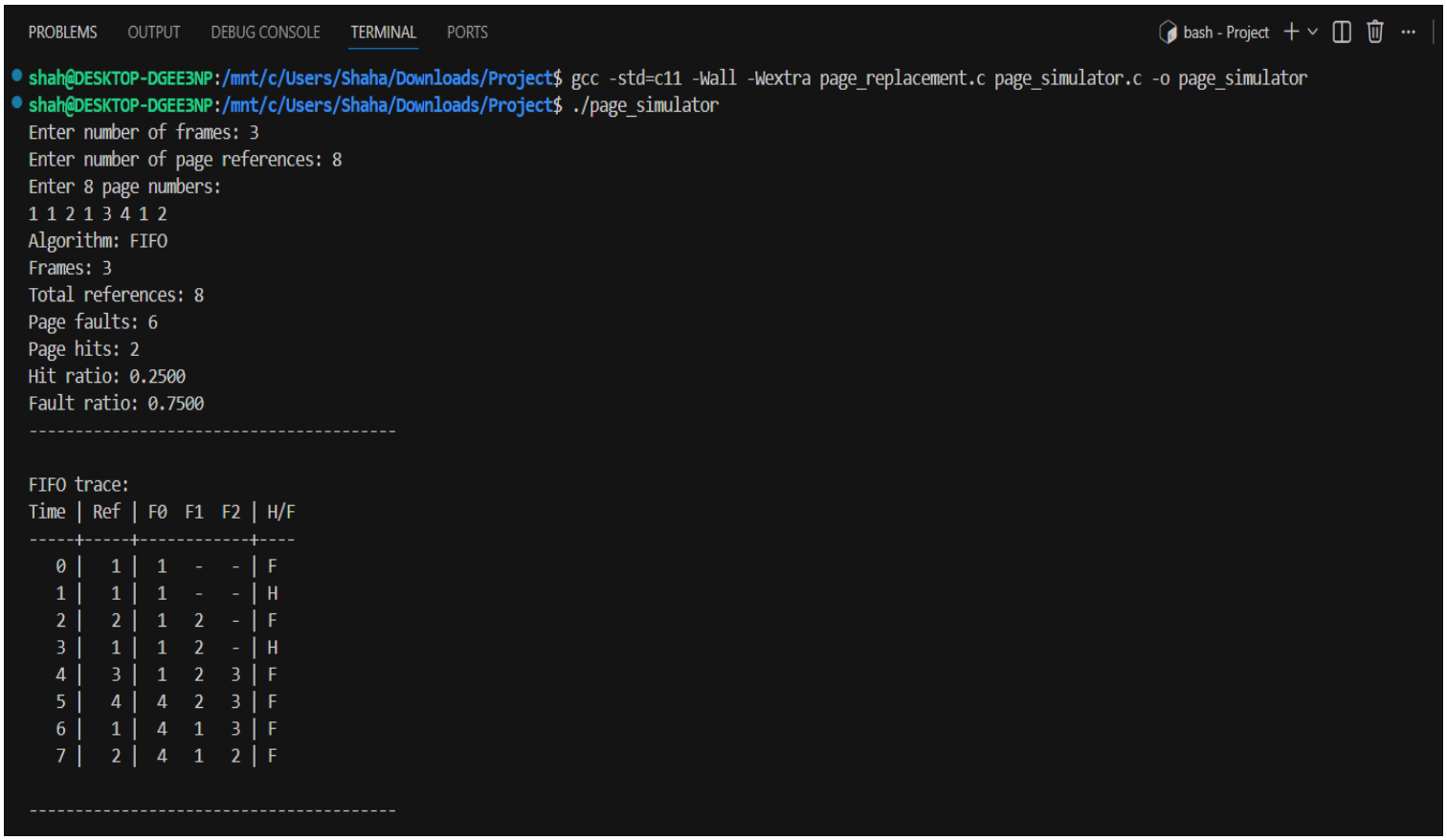
If everything is valid, the driver calls each of the four algorithms in turn. For each one, it prints a summary of the hits and faults and then prints the full trace table. After all four methods finish, the reference array is freed and the program ends cleanly. Throughout the code there are checks for invalid input and failed memory allocations to avoid crashes or strange behavior. The program was tested with several different reference inputs and the results matched what was expected.

Output:

Commands to run the code:

gcc -std=c11 -Wall -Wextra page_replacement.c page_simulator.c -o page_simulator

./page_simulator



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                bash - Project

shah@DESKTOP-DGEE3NP:/mnt/c/Users/Shaha/Downloads/Project$ ./page_simulator

Algorithm: LRU
Frames: 3
Total references: 8
Page faults: 5
Page hits: 3
Hit ratio: 0.3750
Fault ratio: 0.6250
----------------------------------------

LRU trace:
Time | Ref | F0  F1  F2 | H/F
-----+-----+------------+----
   0 |   1 |  1   -   - | F
   1 |   1 |  1   -   - | H
   2 |   2 |  1   2   - | F
   3 |   1 |  1   2   - | H
   4 |   3 |  1   2   3 | F
   5 |   4 |  1   4   3 | F
   6 |   1 |  1   4   3 | H
   7 |   2 |  1   4   2 | F

----------------------------------------
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                bash - Project

shah@DESKTOP-DGEE3NP:/mnt/c/Users/Shaha/Downloads/Project$ ./page_simulator

Algorithm: MIN (Optimal)
Frames: 3
Total references: 8
Page faults: 4
Page hits: 4
Hit ratio: 0.5000
Fault ratio: 0.5000
----------------------------------------

MIN trace:
Time | Ref | F0  F1  F2 | H/F
-----+-----+------------+----
   0 |   1 |  1   -   - | F
   1 |   1 |  1   -   - | H
   2 |   2 |  1   2   - | F
   3 |   1 |  1   2   - | H
   4 |   3 |  1   2   3 | F
   5 |   4 |  1   2   4 | F
   6 |   1 |  1   2   4 | H
   7 |   2 |  1   2   4 | H

----------------------------------------
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash - Project

```
shah@DESKTOP-DGEE3NP:/mnt/c/Users/Shaha/Downloads/Project$ ./page_simulator

Algorithm: Random
Frames: 3
Total references: 8
Page faults: 4
Page hits: 4
Hit ratio: 0.5000
Fault ratio: 0.5000
---------------------------------------

Random trace:
Time | Ref | F0  F1  F2 | H/F
-----+-----+------------+----
   0 |   1 |  1   -   - | F
   1 |   1 |  1   -   - | H
   2 |   2 |  1   2   - | F
   3 |   1 |  1   2   - | H
   4 |   3 |  1   2   3 | F
   5 |   4 |  1   2   4 | F
   6 |   1 |  1   2   4 | H
   7 |   2 |  1   2   4 | H
```